SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1737

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration: 1 Hour**
Total Marks: 50

Annexure A

Scenario Based Assignment

Code of Conduct:

*I **Saai Vardhan D / 192525205** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Saai Vardhan D

Annexure A

Scenario Based Assignment

Name: Saai Vardhan D | Reg. No.: 192525205 | Course: CSA1737 – Artificial Intelligence

Question 1 (10 Marks)

Scenario: A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: a heuristic estimate (straight-line distance), partial real-time updates about blocked paths, and variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

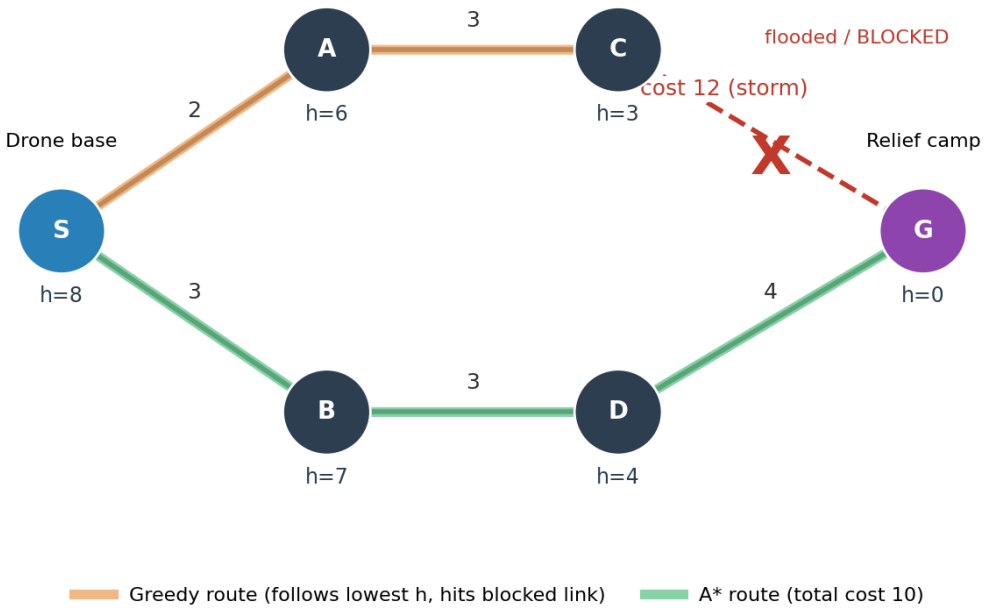
Tasks: (i) Greedy Best-First Search behaviour, strengths and weaknesses; (ii) A* handling and its balance of cost and heuristic; (iii) impact of heuristic accuracy; (iv) effect of dynamic changes; (v) recommendation with justification.

Problem formulation (common to all parts)

Element	Definition for the drone problem
State	Current waypoint (grid cell / junction) of the drone, plus the latest known status of each path.
Initial / Goal	Drone base S / relief camp G where the medicine must be delivered.
Actions	Fly to a neighbouring waypoint along an open path.
Step cost g	Distance × weather factor × terrain factor + risk penalty (battery drain, wind, flood depth).
Heuristic h(n)	Straight-line distance from waypoint n to G (always optimistic because a straight line is the shortest possible route).

To illustrate each algorithm we use the small route graph in Fig. 1. Edge labels are travel costs and h is the straight-line estimate to G. The direct-looking route through C has a very expensive, storm-hit final link C–G (cost 12) that may be blocked by flooding.

Fig. 1 Flood-zone route graph (edge = travel cost, h = straight-line estimate)



(i) Greedy Best-First Search – behaviour, strengths and weaknesses (2 marks)

Evaluation function: f(n) = h(n). Greedy Best-First Search (GBFS) keeps a priority queue (frontier) ordered only by the heuristic and always expands the waypoint that looks closest to the relief camp. It never looks at the cost already spent (g) and never considers how costly the remaining path will really be.

Behaviour in the scenario (Fig. 1): From S the choices are A (h = 6) and B (h = 7). GBFS picks A, then C (h = 3, the smallest in the whole graph). From C the only way forward is the C–G link. If the link is flooded the drone is stuck in a dead end and must backtrack to S and try B–D–G; if it is open but storm-affected, the drone pays cost 2 + 3 + 12 = 17 instead of the optimum 10. The drone therefore moves very fast towards the target but may be led into the most dangerous region.

Strengths

- **Speed:** it expands very few nodes, so a decision is produced in milliseconds – ideal for urgent medicine delivery.
- **Low memory and computation:** only the frontier is stored, which suits small on-board drone processors.
- **Reactive:** when a partial update arrives it can re-pick the most promising neighbour immediately.
- **Works well when the heuristic is close to the true cost** (e.g. clear weather, uniform terrain).

Weaknesses under uncertainty

- **Not optimal:** ignoring g means the cheapest route can be missed (17 vs 10 in Fig. 1).
- **Blind to variable terrain/weather cost:** a straight line over a flooded river looks short but is unflyable or costly.
- **Can walk into dead ends** created by blocked paths and then wastes battery on backtracking.

- **Not complete on graphs with loops** unless a visited set is maintained; it may oscillate between two attractive but blocked waypoints.
- **Over-trusts the heuristic:** when real-time information contradicts the straight-line estimate, GBFS has no cost memory to compensate.

(ii) A* – handling the same situation (2 marks)

Evaluation function: f(n) = g(n) + h(n), where g(n) is the actual accumulated cost from the base (terrain, weather and risk-weighted) and h(n) is the estimated remaining cost. A* always expands the node with the smallest estimated total path cost. The heuristic pulls the search towards the goal (like GBFS) while g penalises expensive or risky legs and keeps the search honest.

Trace on Fig. 1 (h values: S 8, A 6, B 7, C 3, D 4, G 0):

Step	Expand	g	h	f = g + h	Frontier after expansion (f)
1	S	0	8	8	A(8), B(10)
2	A	2	6	8	C(8), B(10)
3	C	5	3	8	B(10), G via C = 17
4	B	3	7	10	D(10), G via C = 17
5	D	6	4	10	G via D = 10 (replaces 17)
6	G	10	0	10	Goal removed from the queue → stop

A* first follows A and C because their f is still low, but as soon as the high cost of C–G is added, the f-value jumps to 17 and the cheaper branch B → D → G (cost 10) overtakes it. **Final path: S → B → D → G with cost 10**, which is optimal. If C–G is reported as blocked, the edge is simply removed from the graph and A* never considers it – no dead end is entered.

How A* balances cost and heuristic

- **g(n) (cost so far)** makes A* cautious: it prefers cheap, safe legs and prunes expensive storm-hit areas.
- **h(n) (heuristic)** makes A* directed: without it the search becomes uniform-cost search (Dijkstra) and explores every direction equally.
- If h is **admissible** (never overestimates) and **consistent**, A* is guaranteed to return the least-cost route. Straight-line distance satisfies both, so A* is optimal here.
- Setting h = 0 gives Dijkstra; setting g = 0 gives GBFS. A* lies between the two extremes.

(iii) Impact of heuristic accuracy on both algorithms (2 marks)

Heuristic quality	Greedy Best-First Search	A*
Perfect (h = true cost)	Goes straight to the goal.	Expands only nodes on the optimal path.
Close but optimistic (admissible)	Fast, usually good, but still no optimality guarantee.	Optimal, with few expansions.
Weak (e.g. h ≈ 0)	Behaves like random/blind search; can wander.	Still optimal, but degrades to Dijkstra – many nodes expanded, slower.
Overestimates (not admissible), e.g. flood detour ignored	Chases wrong direction; may select blocked or risky areas.	May skip the true best route → loses optimality (but may become faster).

Heuristic quality	Greedy Best-First Search	A*
Misleading (straight line crosses a flooded river)	Fails badly – drives the drone to the closest-looking but impassable point.	Recovers because g reveals the real cost; just expands more nodes.

Conclusion: GBFS's quality depends entirely on h, whereas A* uses h only to speed up the search – accuracy affects A*'s efficiency but not its correctness (as long as h is admissible). A practical improvement is a **weather-aware heuristic**: h′(n) = straight-line distance × (minimum weather/terrain multiplier). It is more informed than plain distance and still admissible.

(iv) Effect of dynamic changes (blocked paths) on search performance (2 marks)

Classical GBFS and A* assume a static graph. In a flood zone, paths close and costs change while the drone is flying, so a plan computed at the start can become invalid mid-route.

Aspect	Greedy Best-First	A*
Reaction to a blocked path	Cheap local re-choice of the next best-h neighbour; high chance of dead ends and loops.	Must re-plan; the remaining route is recomputed from the current position, giving a guaranteed best alternative.
Re-planning cost	Very low (few nodes).	Higher, grows with map size; every re-plan may re-expand many nodes.
Route quality after change	Often poor; short-term decisions.	Optimal for the currently known map.
Safety / battery	Backtracking wastes energy.	Fewer wasted moves; can include risk in g.

- **Stale information:** partial updates mean some blocked paths are still unknown; a plan can be optimal on paper but fail in practice. The drone therefore must treat uncertain edges with a risk penalty in g.
- **Performance metrics affected:** number of nodes expanded, re-planning time, total path cost, and number of backtracks all increase with frequent changes.
- **Remedies:** replan from the current position only (receding horizon); use incremental algorithms such as **D* Lite / LPA*** that reuse the previous search and repair only the affected nodes; use **weighted A*** or **anytime A*** (ARA*) to produce a quick first route and improve it while flying.

(v) Recommended algorithm and justification (2 marks)

Recommendation: A* with a weather-aware admissible heuristic, run in a replanning (D* Lite / weighted-A*) mode; Greedy Best-First kept only as an emergency fallback when the time budget is exhausted.

Criterion	GBFS	A*	Verdict
Real-time constraint	Fastest	Slightly slower; made fast with weighted A* ($f = g + 1.3 \cdot h$) and incremental re-planning	Both acceptable; A* tunable
Optimality	No	Yes (admissible h)	A*
Safety / risk	Ignores risk, dead ends likely	Risk and weather encoded in g; avoids high-cost/storm links	A*
Completeness	No (loops/dead ends)	Yes	A*

Criterion	GBFS	A*	Verdict
Adaptability to blocked paths	Reactive but short-sighted	Re-plans optimally; incremental variants reuse work	A*

Justification: Delivering medicine is life-critical, so a route that is safe and reliably reaches the camp is more valuable than one that is merely computed a few milliseconds sooner. A* provides optimal, complete routes and naturally includes terrain and weather cost. Its extra computation is limited by (a) weighted A* to bound delay, (b) re-planning only from the current waypoint, and (c) incremental search (D* Lite). If a hard deadline for computation is reached, the drone temporarily follows GBFS to keep moving and then switches back to A* when new data arrives. This hybrid meets the urgency, optimality and safety requirements together.

Question 2 (10 Marks)

Scenario: A smart city implements an AI system to optimise traffic signal timings across hundreds of intersections. The objective is to minimise total waiting time, fuel consumption and traffic congestion. The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions.

Tasks: (a) formulate as an optimisation problem and explain why traditional search is inefficient; (i) Hill Climbing and its limitations; (ii) local maxima, plateaus, ridges with real-world interpretation; (iii) Simulated Annealing / Genetic Algorithms; (iv) best approach with justification.

(a) Optimisation formulation and why traditional search is inefficient (2 marks)

Element	Formulation
State (solution)	x = vector of timing parameters for all N intersections: cycle length, green-time split of each phase and offset between neighbouring signals, i.e. $x = (c_1, g_1, o_1, \ldots, c_N, g_N, o_N)$.
Neighbourhood / actions	Change one parameter slightly: increase or decrease a green time by δ seconds, shift an offset by a few seconds, swap phase order.
Objective function	Minimise $F(x) = w_1 \cdot W(x) + w_2 \cdot Fu(x) + w_3 \cdot Co(x)$ where W = total waiting time, Fu = fuel consumption, Co = congestion (queue length / density); $w_1+w_2+w_3 = 1$. (For hill-climbing we maximise $-F(x)$.)
Constraints	Minimum green (≈10 s) and maximum cycle (≈120 s); pedestrian crossing time; emergency-vehicle priority; coordination of corridors.
Evaluation	F(x) computed from a traffic simulator or live sensor data (loop detectors, cameras).

Why traditional (uninformed / classical informed) search is inefficient

- **Huge search space:** with 300 intersections, 3 parameters each and about 20 possible values per parameter there are 20^{900} configurations – impossible for BFS, DFS or even A* to enumerate.
- **No single goal state and no path cost:** the task is to find the best configuration, not a path; the goal test of classical search is meaningless and no admissible heuristic exists.
- **Memory:** BFS/A* store exponentially many nodes; the problem needs a constant-memory method.
- **Dynamic environment:** traffic changes every few minutes, so a precomputed optimum becomes obsolete; the method must be anytime and re-runnable.
- **Expensive evaluation and coupling:** each evaluation needs a simulation, and neighbouring intersections interact (a green wave), so variables cannot be optimised independently.

Hence **local-search / metaheuristic optimisation** (Hill Climbing, Simulated Annealing, Genetic Algorithms) is appropriate: they keep only the current solution(s), improve it iteratively, and can be stopped any time.

(i) Hill Climbing in this scenario and its limitations (2 marks)

Hill climbing starts from the current signal plan and repeatedly moves to a neighbouring plan with a better objective value, stopping when no neighbour is better.

```
current ← current signal plan x
```

```
loop:
    neighbours ← plans obtained by changing one green time / offset by ±δ
    best ← neighbour with the lowest F (highest −F)
    if F(best) ≥ F(current): return current      # no improvement
    current ← best
```

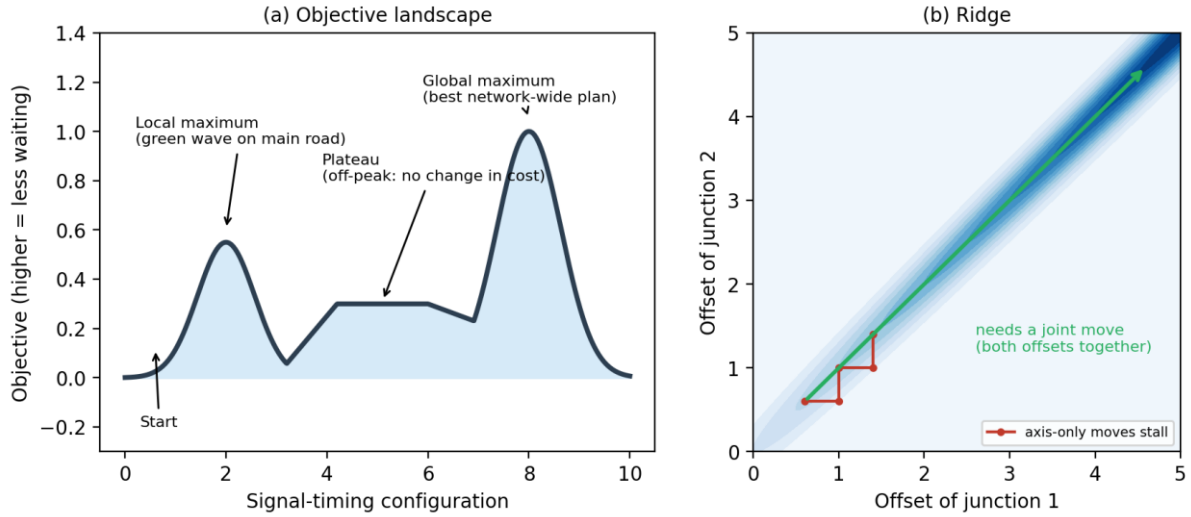
- **Steepest-ascent:** evaluates all neighbours, picks the best. **First-choice / stochastic:** takes the first (or a random) improving neighbour – cheaper for huge neighbourhoods. **Random-restart:** repeats from several initial plans.
- **Advantages:** O(1) extra memory, very fast, can run continuously and adapt to live traffic (just keep climbing from the current plan).

Limitations

- **Local maxima:** stops at the first peak even if a better plan exists.
- **Plateaus / shoulders:** flat regions give no direction; search wanders or halts.
- **Ridges:** improvement needs several variables to change together, but hill climbing changes one at a time.
- **Result depends on starting plan;** no way to go downhill temporarily; sensitive to noisy measurements; no guarantee of optimality.

(ii) Local maxima, plateaus and ridges – real-world interpretation (2 marks)

Fig. 2 Hill-climbing pitfalls in traffic-signal optimisation



Problem	What it looks like in the objective landscape	Real-world traffic interpretation
Local maximum	A peak higher than all neighbours but lower than the global peak.	The main road has a perfect “green wave”. Any single tweak makes waiting time worse, so the algorithm stops, although giving some priority to the cross-street corridor would reduce total city-wide delay.
Plateau	A flat region – neighbouring plans have the same value.	Late at night, traffic is so light that changing green times by a few seconds does not change waiting time. The algorithm has no gradient and wanders or terminates.

Problem	What it looks like in the objective landscape	Real-world traffic interpretation
Ridge	A narrow high strip running diagonally; moves along any one axis go downhill.	To keep a green wave, the offsets of two adjacent junctions must change together. Changing only one breaks synchrony and the objective drops, so single-variable moves are rejected.

(iii) How Simulated Annealing or Genetic Algorithms overcome these limitations (2 marks)

Simulated Annealing (SA)

SA imitates the slow cooling of metal. A random neighbour is generated; if better it is always accepted, and if worse (ΔE = increase in cost) it is accepted with probability **P = e^(–ΔE/T)**. The temperature T starts high (accepts many worse moves → exploration) and is reduced gradually, e.g. T ← αT with α = 0.95–0.99 (focus on exploitation).

```
T ← T₀ ; current ← initial plan
while T > T_min:
    next ← random neighbour of current ; ΔE ← F(next) − F(current)
    if ΔE < 0 or random() < exp(−ΔE / T): current ← next
    T ← α · T
```

- Accepting occasional worse plans lets SA **climb out of local maxima and cross plateaus**; with slow cooling it converges to the global optimum with probability → 1.
- Still needs only one solution in memory; simple to implement; can be "re-heated" when traffic pattern changes.

Genetic Algorithm (GA)

A GA maintains a **population** of complete signal plans (chromosomes). Fitness = 1 / F(x). Each generation applies **selection** (better plans reproduce), **crossover** (combine timing sections of two parents, e.g. take the north corridor from plan 1 and the east corridor from plan 2) and **mutation** (random small changes), with **elitism** to keep the best plan.

- The population explores many regions at once, so it avoids being trapped in a single local peak.
- Crossover can change several variables together, which solves the **ridge** problem; mutation randomness handles **plateaus**.
- Evaluations of the population can be run in parallel, matching a city-scale simulator; seeding the population with yesterday's best plan gives fast adaptation.

(iv) Recommended approach and justification (2 marks)

Criterion	Hill Climbing	Simulated Annealing	Genetic Algorithm
Escapes local maxima	No	Yes (probabilistic)	Yes (population diversity)
Handles ridges	No	Partly	Yes (crossover)
Scalability (hundreds of junctions)	High	High	High with parallel evaluation / decomposition
Adaptability to changing traffic	Good for tiny tuning	Good (re-heat)	Very good (warm-start population)

Criterion	Hill Climbing	Simulated Annealing	Genetic Algorithm
Memory / compute	Lowest	Low	Highest
Solution quality	Lowest	High	Highest

Recommendation: a hybrid – Genetic Algorithm for the global search, followed by Simulated Annealing / Hill Climbing for fast local fine-tuning.

- **Scalability:** divide the city into zones or corridors; each zone is optimised by its own GA/SA in parallel and boundary offsets are reconciled – the cost grows roughly linearly with the number of zones rather than exponentially.
- **Adaptability:** every few minutes the previous best plans seed the new population (warm start), and a light SA/hill-climbing step reacts instantly to sudden changes such as an accident or event traffic.
- **Quality and robustness:** GA diversity avoids local maxima and ridges, SA/hill-climbing polishes the result cheaply, and the anytime nature means a valid plan is always available.

Question 3 (10 Marks)

Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks: (i) why an online search agent is needed; (ii) partial observability and dynamic environments; (iii) building and updating the internal model; (iv) comparison with uninformed and informed search; (v) most suitable approach.

(i) Why an online search agent instead of offline search (2 marks)

Offline search computes a complete solution before acting, which requires a full and correct model of the environment (map, action results, costs). **Online search** interleaves computation and action: the agent plans one step, acts, senses, and plans again. The rover needs the second approach because:

- **The map is unknown:** the result of an action (what lies in the next cell) is discovered only by performing it – an offline planner has nothing to search through.
- **Exploration is part of the task:** the rover must learn the world while pursuing the goal (exploration vs exploitation).
- **Safety and irreversibility:** a wrong step (falling into a crater) cannot be undone; the agent must only take steps from which it can return.
- **Communication delay:** Earth–Mars signal delay of roughly 4–24 minutes makes remote control impossible, so decisions must be autonomous and immediate.
- **Limited resources:** battery and compute are limited, so planning is done incrementally, not for the whole mission.

Feature	Offline search	Online search
Knowledge of environment	Complete model needed	Unknown / discovered while moving
Planning and acting	Plan first, then act	Interleaved
Handles surprises	Poorly (must re-run)	By design
Typical algorithms	BFS, UCS, A*	Online DFS, LRTA*, D* Lite
Evaluation metric	Solution cost	Competitive ratio (actual cost ÷ cost if map were known)

(ii) Challenges of partial observability and dynamic environments (2 marks)

Partial observability

- **Limited sensor range and occlusion:** the camera/lidar sees only a few metres; rocks hide terrain behind them, so unknown cells may hide hazards.
- **Sensor noise:** dust, shadows and low light cause false positives (a shadow seen as a crater) or false negatives (a hidden rock).
- **State ambiguity and localisation:** wheel slip and the absence of GPS make the rover unsure of its exact position – it must maintain a belief state (probability distribution over positions) using SLAM.

- **Dead ends:** it may enter a region from which the way back is blocked; safe exploration must guarantee a return path.

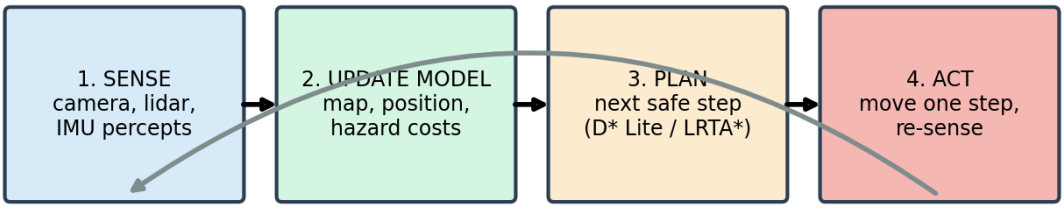
Dynamic environment

- Dust storms, drifting sand and rolling rocks change terrain after it has been mapped; previous map knowledge can become outdated.
- Lighting, temperature and slope conditions change the cost of movement (wheel slip, power drain).
- Energy budget and time (solar charging) change continuously, so the best plan changes as well.

Consequences for search: plans must be short-term and revisable, unknown areas need a cautious cost (treated as risky until sensed), and the agent should maintain uncertainty-aware models instead of a fixed state graph.

(iii) How the rover builds and updates its internal model (2 marks)

Fig. 3(a) Online search-agent cycle of the rover

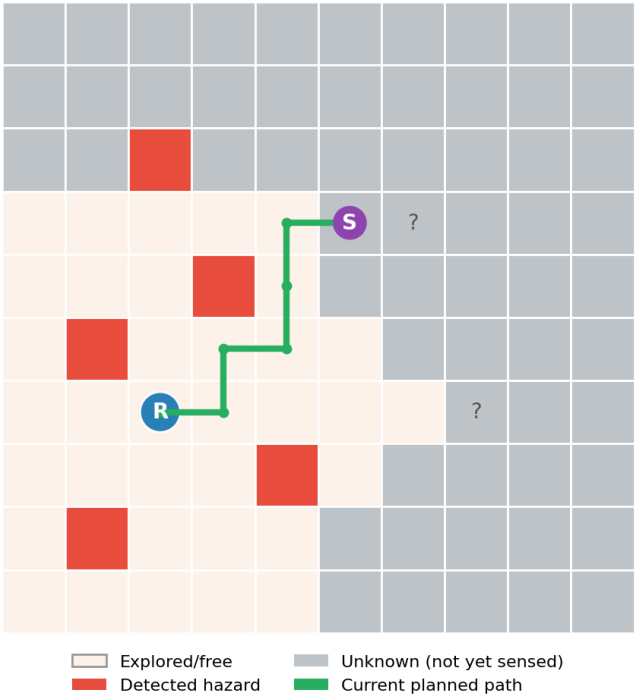


repeat until sample collected (safety monitor can override at every cycle)

The rover maintains a model of the world that is refined every cycle (Fig. 3(a)):

- **Perception:** stereo cameras, lidar and IMU give a local percept (visible cells and slopes).
- **Occupancy grid map:** the area is divided into cells, each storing the probability of being free, occupied/hazard or unknown. It is updated with Bayes' rule (log-odds): a cell seen several times as a rock becomes "occupied" with high confidence. Unknown cells are initialised as unknown (not safe).
- **Localisation (SLAM):** the rover's pose is estimated and corrected by matching new observations with the existing map.
- **Transition/result table:** the agent stores RESULT(s, a) = s′ and the observed step cost for every action tried; this is the online agent's learned model of the world.
- **Heuristic table H(s):** in LRTA* the estimated cost-to-goal of each visited cell is updated as H(s) ← min over actions a of [c(s, a, s′) + H(s′)], so the agent learns better estimates the more it explores.
- **Frontier list:** boundaries between explored and unexplored cells are kept as candidate exploration targets.

Fig. 3(b) Rover's internal map (R = rover, S = sample site)



After each move, newly sensed hazards (red) are added, the explored area expands, and the planned path is recomputed from the rover's current cell.

(iv) Comparison of online search with uninformed and informed search (2 marks)

Criterion	Uninformed (BFS, DFS, UCS)	Informed (Greedy, A*)	Online search (online DFS, LRTA*, D* Lite)
Map knowledge needed	Complete	Complete (for graph + h)	None initially; learned while exploring
Uses heuristic?	No	Yes	Yes – and improves it by learning
Plans before acting	Yes	Yes	No – one step at a time
Handles unknown/changing terrain	No	No (must restart)	Yes
Backtracking	Logical (in memory)	Logical	Physical – must really move back, so safety/cost matters
Optimality	BFS/UCS optimal	A* optimal (admissible h)	Not guaranteed; measured by competitive ratio
Memory	High (BFS) or low (DFS)	High	Moderate – stores map and H table
Fit for Mars rover	Poor	Poor-to-moderate (after map known)	Best

Uninformed and informed strategies assume that the whole state space and all action outcomes are available before the search begins. In the rover scenario, expanding a node in memory is not the same as moving there: only physical movement reveals new information. Online agents are the only class designed for this.

(v) Most suitable approach considering uncertainty, adaptability and safety (2 marks)

Recommendation: an online incremental heuristic search – D* Lite (or LRTA*) over a continuously updated occupancy-grid map, with frontier-based exploration and a safety layer.

- **Uncertainty:** unknown cells are treated as possibly blocked with an added risk cost; belief about the rover pose (SLAM) is used; sensor noise is reduced by repeated observations.
- **Adaptability:** D* Lite starts as A* from the goal to the rover and, when new hazards are sensed, repairs only the affected part of the previous solution, so re-planning is fast enough for real time. LRTA* learns improved H values automatically.
- **Safety:** a safety layer enforces a minimum clearance from hazards and slope limits, forbids moves into cells whose safety is unknown, keeps a guaranteed return path (safely explorable), preserves an energy reserve to return to base, and can trigger a stop-and-wait fallback.
- **Efficiency:** exploration is biased toward the sample goal by the heuristic, so the rover does not waste energy on uniform exploration.

Together this gives a rover that acts in real time, improves its model continuously, and never sacrifices safety for speed.

Question 4 (10 Marks)

Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: no student should have overlapping exams, limited number of exam halls, certain subjects must be scheduled before others, faculty availability constraints, and special accommodations for some students. The complexity increases as the number of courses and students grows.

Tasks: Formulate as a CSP; (i) define variables, domains and constraints; (ii) backtracking search; (iii) constraint propagation (forward checking); (iv) real-world challenges and best strategy for scalability.

(i) CSP formulation – variables, domains and constraints (3 marks)

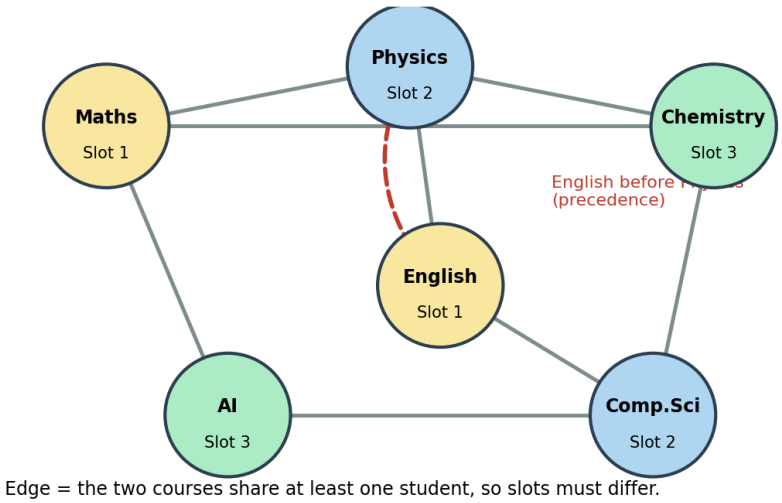
A Constraint Satisfaction Problem is defined by a triple ⟨X, D, C⟩: variables X, a domain D for each variable and a set of constraints C. A solution is a complete and consistent assignment.

Component	Definition	Explanation
Variables X	Slot_c and Hall_c for every course c (c = 1…n); optionally Invigilator_c	Each course needs a time slot and a room. (Slot variables are the main decision variables.)
Domain of Slot_c	D(Slot_c) = {t_1, t_2, … t_T}, T = exam days × sessions per day (e.g. 10 days × 2 = 20 slots)	The possible time periods; may be reduced by faculty availability.
Domain of Hall_c	D(Hall_c) = {halls with capacity ≥ enrolment(c)} (or a set of halls if the exam is split)	Prevents assigning a course to a hall that is too small.

Constraints

#	Type	Formal constraint	Meaning / explanation
C1	Hard, binary	Slot_a ≠ Slot_b for every pair (a, b) with a common student	No student has overlapping exams (the conflict graph in Fig. 4).
C2	Hard, global	Number of courses in slot t ≤ number of halls; and (Slot_a = Slot_b) ⇒ Hall_a ≠ Hall_b; Σ enrolment in hall ≤ capacity	Limited exam halls: two exams cannot share the same hall at the same time, and capacities are respected.
C3	Hard, binary	Slot_a < Slot_b (e.g. Maths-I before Maths-II; English before Physics)	Precedence between subjects.
C4	Hard, unary	Slot_c ∈ Available(faculty of c); invigilators not in two halls at once	Faculty availability.
C5	Hard/soft, unary + binary	Special-needs students: hall must be accessible/ground floor, extra time ⇒ longer slot, and no more than 2 exams a day (soft) or a gap between exams	Special accommodations.

Fig. 4 Exam constraint graph with a valid 3-slot assignment



Edge = the two courses share at least one student, so slots must differ.

In Fig. 4 each node is a course (variable), an edge is constraint C1 (shared students), the colour is the assigned slot (the domain value), and the red dashed arrow is a precedence constraint C3. The 3-colouring shown is a valid solution, which shows that exam timetabling is essentially **graph colouring** (NP-hard) with additional constraints.

(ii) How backtracking search works in this scenario (2 marks)

Backtracking search is depth-first search over partial assignments: choose an unassigned variable, try each value from its domain, check the constraints against the current partial assignment, recurse if consistent, and **backtrack** (undo and try the next value or go back to the previous variable) when a variable has no consistent value left.

```
function BACKTRACK(assignment):
    if assignment is complete: return assignment
    var ← SELECT-UNASSIGNED-VARIABLE()
    for value in ORDER-DOMAIN-VALUES(var):
        if value consistent with assignment:
            add {var = value}
            result ← BACKTRACK(assignment)
            if result ≠ failure: return result
            remove {var = value}                # backtrack
    return failure
```

Example (Fig. 4, three slots S1, S2, S3, with extra rule 'English before Physics'):

Step	Assignment	Result
1	Maths = S1	Consistent
2	Physics = S1 ✗ (conflicts with Maths) → S2	Consistent
3	Chemistry = S1 ✗, S2 ✗ → S3	Consistent
4	Comp.Sci = S1 (adjacent: Chemistry S3 only)	Consistent so far
5	AI = S1 ✗ (Maths, Comp.Sci), S2	Consistent
6	English: S1 ✗ (Comp.Sci), S2 ✗ (Physics), S3 ✗ (must be before Physics = S2)	**Dead end – no value → backtrack**

Step	Assignment	Result
7	Back to AI: try S3 → English still has no value → backtrack to Comp.Sci	Wasted work (AI is unrelated to the failure)
8	Comp.Sci = S2 → AI = S3 → English = S1	**Complete consistent solution:** Maths S1, Physics S2, Chemistry S3, Comp.Sci S2, AI S3, English S1

The example also shows the weakness of **chronological backtracking**: it discovered the failure late and retried irrelevant variables (AI). This motivates constraint propagation. Variable ordering heuristics help too: **MRV** (choose the variable with the fewest legal values), **degree heuristic** (most constraints on others) and **LCV** (try the value that rules out the fewest options for neighbours).

(iii) How constraint propagation (forward checking) improves efficiency (2.5 marks)

Forward checking (FC): after assigning a variable, remove from the domains of its unassigned neighbours all values inconsistent with that assignment. If any domain becomes empty, backtrack immediately – long before reaching that variable.

Same example with forward checking. Initial domains: Physics {S1,S2,S3}, English {S1,S2,S3}, etc. After applying the precedence rule English < Physics = S2 → **English = {S1}**. When Comp.Sci = S1 is assigned, FC removes S1 from its neighbours' domains, so **English = { } (empty)** → failure is detected immediately, without assigning AI at all. The algorithm backtracks directly to Comp.Sci and tries S2.

	Plain backtracking	Backtracking + forward checking
Failure detected	At step 6, after assigning AI	At step 4, right after Comp.Sci = S1
Variables reassigned unnecessarily	AI re-tried (extra work)	None
Nodes explored	More	Fewer

- **Arc consistency (AC-3 / MAC):** goes further than FC; it removes any value of X that has no compatible value in a neighbour Y, and propagates repeatedly. E.g., a precedence chain A < B < C with only 3 slots forces A = S1, B = S2, C = S3 before any search begins.
- **Benefit:** the search tree is pruned early, the number of backtracks drops dramatically (often by orders of magnitude), and with MRV, a failing variable is detected first ("fail-first" principle).

(iv) Real-world challenges and best strategy for scalability (2.5 marks)

Challenge	Why it matters	Handling
Problem size	Thousands of courses and tens of thousands of students; graph colouring is NP-hard.	Constraint propagation + heuristics; decomposition.
Over-constrained instances	A fully feasible timetable may not exist (e.g. too few halls).	Treat some constraints as soft; use weighted/partial CSP and minimise violation cost.
Soft preferences	Gaps between exams, avoiding back-to-back, fairness for students.	Cost function + local search optimisation.
Late changes	Hall closed, faculty unavailable, a re-evaluation request.	Incremental repair – re-solve only the affected variables (min-conflicts).

Challenge	Why it matters	Handling
Data quality	Inaccurate enrolment data produces wrong conflicts.	Validation and conflict reports before solving.
Special accommodations	Accessibility, extra time and separate rooms add many unary constraints.	Encode as domain filters early (pruning).

Recommended strategy: a hybrid approach.

- **Step 1 – Preprocessing:** build the conflict graph; apply node and arc consistency (AC-3) with unary constraints (faculty, accessibility) to shrink domains.
- **Step 2 – Systematic search:** backtracking with MRV + degree + LCV and forward checking / MAC, plus conflict-directed backjumping, to get a feasible timetable satisfying all hard constraints.
- **Step 3 – Optimisation:** run local search (min-conflicts, tabu search or simulated annealing) to reduce soft-constraint penalties, and to repair the timetable when changes occur.
- **Step 4 – Decomposition:** split by department/day cluster and solve sub-problems in parallel; larger-degree courses (e.g. common subjects) are scheduled first.

Justification: pure backtracking is complete but explodes in time for large instances, while pure local search scales but cannot guarantee that hard constraints are satisfied. The hybrid guarantees feasibility, scales to university size, and supports quick re-scheduling.

Question 5 (10 Marks)

Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning). A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, make decisions within strict time limits, evaluate complex game states with multiple possible moves, and the decision tree is extremely large, making computation expensive.

Tasks: (i) how Minimax models the problem; (ii) computational challenges; (iii) Alpha-Beta pruning; (iv) evaluation function; (v) balancing optimality and real-time performance.

(i) How the Minimax algorithm models this problem (2 marks)

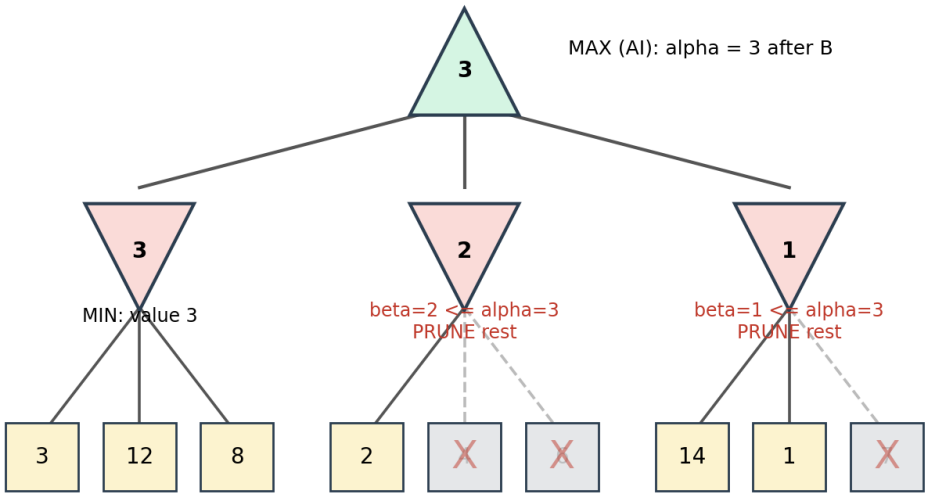
The game is treated as a **two-player, zero-sum, adversarial search problem**: the AI is **MAX** (maximises its score) and the human opponent is **MIN** (assumed to play optimally to minimise the AI's score). The problem is defined by:

Component	In the RTS game
Initial state	Current game snapshot: units, positions, resources, buildings, fog-of-war knowledge.
PLAYER(s)	Whose turn it is: MAX (AI) or MIN (human) – real-time play is discretised into decision ticks.
ACTIONS(s)	Legal commands: move/attack/build/train/research (the branching factor b).
RESULT(s, a)	Game state after the action.
TERMINAL-TEST(s)	Win, loss or time/depth limit reached.
UTILITY(s)	+1 / 0 / –1 for win / draw / loss, or an evaluation score for non-terminal depth-limited states.

Minimax value: MINIMAX(s) = UTILITY(s) if s is terminal; max over actions of MINIMAX(RESULT(s,a)) if PLAYER(s) = MAX; min over actions of MINIMAX(RESULT(s,a)) if PLAYER(s) = MIN. The values are computed depth-first from the leaves upward: MAX nodes take the largest child value, MIN nodes the smallest.

Example (Fig. 5): the AI has three moves (B, C, D). For each, the opponent's best reply gives values B = min(3, 12, 8) = 3, C = min(2, 4, 6) = 2 and D = min(14, 1, 7) = 1. The AI chooses **B** with value max(3, 2, 1) = **3** – the best outcome guaranteed against the best human play.

Fig. 5 Minimax tree with Alpha-Beta pruning



Leaf utilities (human player's best reply is MIN). Crossed leaves are never evaluated: 7 of 9 leaves examined.

(ii) Computational challenges of large game trees (2 marks)

Minimax explores the entire tree to depth d: **time $O(b^d)$**, space O(b·d) for depth-first implementation.

Branching factor b	Depth d	Nodes ≈ b^d
30	4	810,000
30	6	729,000,000
30	8	6.6×10^{11}

- **Exponential growth:** a small increase in depth multiplies the work by b; hundreds of units in RTS games give branching factors of thousands (chess ≈ 35).
- **Strict time limits:** the AI must answer in a few milliseconds per frame; exhaustive search is impossible.
- **Real-time and simultaneous moves:** both players act continuously, not alternately; the tree must be discretised.
- **Imperfect information (fog of war)** and randomness add chance nodes, further enlarging the tree.
- **Costly state evaluation:** each node requires simulating the game and computing a complex evaluation.
- **Redundant states:** different move orders reach the same state (transpositions), wasting effort.

Therefore the search must be **depth-limited, pruned and guided** rather than exhaustive.

(iii) Alpha-Beta pruning – efficiency with detailed reasoning (2 marks)

Alpha-Beta pruning computes the same decision as Minimax but skips branches that cannot influence it. Two bounds are maintained: **α** = best (highest) value MAX can already guarantee along the current path; **β** = best (lowest) value MIN can already guarantee. **Prune the remaining children whenever α ≥ β.**

Step-by-step on Fig. 5 (root: α = –∞, β = +∞):

Step	Node visited	Update	Action
1	B – leaves 3, 12, 8	β becomes 3 and stays 3 → B = 3	All leaves necessary. Root α = 3.
2	C – first leaf 2	β = 2, now β (2) ≤ α (3)	**Prune leaves 4 and 6**: MIN can force C ≤ 2, so MAX will never choose C.
3	D – leaves 14, 1	β = 14 then β = 1 ≤ α (3)	**Prune leaf 7**: D ≤ 1 < 3.
4	Root	max(3, 2, 1) = 3	Same answer as Minimax: choose B, with 7 of 9 leaves examined.

- **Correctness:** pruning never changes the minimax value or the chosen move.
- **Efficiency:** worst case $O(b^d)$ (no pruning); with perfect move ordering O(b^(d/2)) – so the AI can search **twice as deep** in the same time; with random ordering about O(b^(3d/4)).
- **Move ordering** is therefore essential: try captures, strong attacks and previously good moves (killer/history heuristics, transposition table) first so cut-offs happen early.

(iv) Designing an evaluation function and justifying the factors (2 marks)

Because the game tree cannot be searched to the end, the search stops at a depth limit and non-terminal states are scored with a heuristic **evaluation function** (zero-sum, symmetric: positive = good for AI):

Eval(s) = w_1·Army + w_2·Economy + w_3·MapControl + w_4·Tech + w_5·BaseSafety + w_6·Vision, $\Sigma w_i = 1$.

Each factor is normalised to [–1, 1] as (AI value – opponent value) / (AI value + opponent value).

Factor	How measured	Why included	Weight
Army strength	Total health × attack of units, weighted by unit type counters	Directly determines the ability to win fights	0.30
Economy	Number of workers, resource income, bank	Long-term production power	0.25
Map control	Share of resource points/territory controlled	Controls expansion and denies enemy income	0.15
Technology	Upgrades and advanced units unlocked	Future strength	0.10
Base safety	Defensive structures, hit points, enemy units near base	Avoids a sudden loss	0.10
Vision / scouting	Fraction of map observed, enemy buildings known	Reduces fog-of-war uncertainty	0.10

Numerical example: Army +0.4, Economy +0.2, MapControl –0.1, Tech 0, BaseSafety +0.3, Vision +0.2 gives Eval = 0.30(0.4) + 0.25(0.2) + 0.15(–0.1) + 0.10(0) + 0.10(0.3) + 0.10(0.2) = 0.12 + 0.05 – 0.015 + 0 + 0.03 + 0.02 = **+0.205** – the AI is moderately ahead.

Design principles: the function must be (a) very fast to compute because it is called at every leaf, (b) strongly correlated with the chance of winning, (c) consistent with terminal utilities (wins ≫ any heuristic value), and (d) tuned – weights are learned by self-play, TD-learning or regression on game logs rather than guessed.

(v) Strategy for balancing optimality and real-time performance (2 marks)

Recommended strategy: iterative-deepening Alpha-Beta with strong move ordering, a transposition table, a depth-limited evaluation function and strict time management.

- **Iterative deepening with a time budget:** search depth 1, 2, 3 … until the time limit; always keep the best move of the last completed depth, so a valid answer exists at any moment (anytime behaviour). The shallower searches also improve move ordering for deeper ones.
- **Alpha-Beta + move ordering + transposition table:** maximises depth reachable within the budget and avoids re-searching repeated states.
- **Depth limit + quiescence search:** apply the evaluation function only at quiet states (not in the middle of a battle) to avoid misleading scores.
- **Selective pruning / beam search:** examine only the top-k promising moves (forward pruning) – small loss of optimality for large gains in speed.
- **Action abstraction:** plan with macro-actions ("attack lane 2", "expand base") at a strategic level and let low-level scripts execute – this cuts the branching factor drastically.
- **Dynamic time allocation:** spend more time in critical situations (battles) and less in routine ones; run search on a separate thread so frame rate is unaffected.
- **Alternative/hybrid:** for very large branching factors, Monte-Carlo Tree Search (MCTS) can replace or complement minimax.

Justification: exact optimal play is impossible within the time limit, but this approach gives the best available move at every instant (anytime), searches as deep as the budget allows (pruning), and is evaluated with a well-tuned heuristic. It therefore keeps the AI strong and responsive – the right trade-off between **optimality** and **real-time performance**.